Integer division

Document number:
P3724R0

Date:
2025-06-04

Audience:
LEWG, SG6

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3724/github

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/intdiv.cow

---

ABSTRACT: C++ currently only offers truncating integer division in the form of the `/` operator. I propose standard library functions for computing quotients and remainders with other rounding modes.

Contents

§ 1. Introduction

C++ currently only offers truncating integer division in the form of the `/` operator. However, other rounding modes have various use cases too, and implementing these as the user can be surprisingly hard, especially when integer overflow needs to be avoided, and negative inputs are accepted.

Furthermore, since the `/` operator rounds towards zero, the `%` (remainder) operator may yield negative results (specifically, when the dividend is negative). In modular arithmetic and various other use cases, this is undesirable, and a negative remainder may be surprising in general.

Therefore, I propose a set of standard library functions which implement a variety of rounding modes when computing quotients and remainders. Such a feature was previously part of [P0105R1] and the Numerics TS [P1889R1], but was eventually abandoned by the author.

NOTE: Terminology refresher for division, where *round* is some rounding function:

$$\text{quotient} = \mathit{round}\left(\frac{\text{dividend}}{\text{divisor}}\right) + \text{remainder}$$

Your browser requires MathML support to view the equation above.

1.1. Existing practice

To better put this proposal into context, it is relevant to know what rounding modes are usually supported by programming languages and hardware.

1.1.1. Language support for integer division

Language	Quotient	Remainder	Rounding
C and C++	`/`	`%`	to zero
D	`/`	`%`	to zero
Objective-C	`/`	`%`	to zero
C#	`/`	`%`	to zero
Java	`/`	`%`	to zero
Rust	`/`	`%`	to zero
Go	`/`	`%`	to zero
Swift	`/`	`%`	to zero
Scala	`/`	`%`	to zero
OCaml	`/`	`mod`	to zero
Perl	`int($a/$b)`	`%`	to zero
GLSL	`/`	N/A	to zero
JavaScript	`/`	`%`	to zero (for `BigInt` operands)
Python	`//`	`%`	to $-\infty$
Lua	`//`	`%`	to $-\infty$
R	`%/%`	`%%`	to $-\infty$
Dart	`~/`	`%`	to $-\infty$
Haskell	`quot`, `div`	`rem`, `mod`	• `quot` and `rem` to zero • `div` and `mod` to $-\infty$
Ada	`/`	`rem`, `mod`	• `/` and `rem` to zero • `mod` to $-\infty$
Fortran	`/`	`mod`, `modulo`	• `/` and `mod` to zero • `modulo` to $-\infty$
CSS	N/A	`rem()`, `mod()`	• `rem()` to zero • `mod()` to $-\infty$
Kotlin	`/`, `rem()`	`%`, `mod()`	• `/` and `rem()` to zero • `mod()` to $-\infty$
Julia	`div(x,y,r)`	`rem(x,y,r)`	depending on `RoundingMode r`: • to zero (default), • away from zero,

			• to $+\infty$ or $-\infty$, or • to nearest (ties to even)

NOTE: When a division rounds towards zero, the remainder has the dividend (left operand) sign. When a division rounds towards $-\infty$, the remainder has the divisor (right operand) sign.

1.1.2. Hardware support for integer division

Architecture	Type	Quotient	Remainder	Rounding
x86 and x86_64	CPU	`idiv`, `div`	`idiv`, `div`	to zero
ARMv7-A and newer	CPU	`sdiv`, `udiv`	N/A	to zero
RISC-V	CPU	`div`, `divu`	`rem`, `remu`	to zero
PowerPC	CPU	`divw`, `divwu`	N/A	to zero
MIPS	CPU	`div`, `divu`	`div`, `divu`, `rem`, `remu`	to zero
AVR (8-bit), PIC16, etc.	CPU	N/A	N/A	N/A
WebAssembly	VM	`div_s`, `div_u`	`rem_s`, `rem_u`	to zero
LLVM IR	VM	`sdiv`, `udiv`	`srem`, `urem`	to zero
Java Bytecode	VM	`idiv`	`irem`	to zero

NOTE: For unsigned division like `div` on x86, there is no distinction between rounding to zero and rounding to $-\infty$, which is why everything is listed as "to zero" for simplicity.

While every architecture rounds to zero, there there are major differences in how the remainder is obtained:

- On x86_64 and MIPS, the quotient and remainder are computed simultaneously, although MIPS also supports separate computation of the remainder.
- On other architectures, even if there is support for computing the remainder separately, if both the quotient and remainder are needed, it is faster to compute the remainder using the quotient of the division.
- In some cases, the remainder can *only* be computed using the quotient.

NOTE: Given two integers x and y, the quotient $q = \left\lfloor \frac{x}{y} \right\rfloor$ (or rounded otherwise to an integer), the remainder of the division is equal to $x - y \times q$.

Only if the division rounds towards zero can this also be translated literally into a multiplication and subtraction, without the possibility of overflow. This makes rounding towards zero *by far* the most useful rounding mode in computing, and it is no coincidence that every architecture implements it.

§ 2. Motivation

Rounding modes other than rounding towards zero are commonly useful. An extremely common alternative is rounding towards $-\infty$, which is the rounding mode of the division operator in some other languages (§1.1.1. Language support for integer division). See below for some motivating examples.

EXAMPLE: A common problem is to compute how many chunks/buckets/blocks of a fixed size are needed to fit a certain amount of elements, which involves a division which rounds towards $+\infty$.

```
const int bucket_size = 1000;
int elements = 100;

int buckets_required = elements / bucket_size; // WRONG, zero
int buckets_required = std::div_to_inf(elements, bucket_size); // OK, on
```

EXAMPLE: A common problem is to compute which chunk/bucket/block an element falls into. This requires division which rounds towards $-\infty$.

```
const int bucket_size = 1000;
int a = 10;
int b = -10;

int a_bucket = a / bucket_size; // OK, zero
int b_bucket = b / bucket_size; // WRONG, also zero

int a_bucket = std::div_to_neg_inf(a, bucket_size); // OK, zero
int b_bucket = std::div_to_neg_inf(b, bucket_size); // OK, -1
```

Note that with truncating division, the zero-bucket would contain all elements in [-999, 999], which would make it larger than any other bucket.

While the examples are somewhat abstract, they appear in vast amounts of concrete problems. For example, we need to know how many blocks of memory must be allocated to hold a certain amount of bytes, do interval arithmetic, fixed-point arithmetic with rounding of choice, etc. etc.

§ 2.1. Is this not trivial for the user to do?

At first glance, it would seem trivial to change rounding modes by making slight adjustments to `/`. However, doing so with correct output, high performance, and without introducing more undefined behavior than `x / y` already has, is surprisingly hard.

There is an *ocean* of examples where C and C++ users have gotten this wrong. A few droplets are listed below.

BUG: At the time of writing, the *first* Google search result for "c++ ceiling integer division" yields [StackOverflowCeil]. Almost every answer does not permit signed integers, gives wrong results, or has undefined behavior for certain inputs (not counting division by zero or `INT_MIN / -1`).

For example, one answer with 67 upvotes (at the time of writing) attempts to implement ceiling (rounded towards $+\infty$) integer division as follows:

```
q = (x % y) ? x / y + 1 : x / y;
```

The quotient `q` would be `1`, not `0` for inputs `x = -1` and `y = 2`, which is obviously wrong because it rounds `-0.5` up to `1`, skipping zero.

To be fair, the answer is correct for positive integers, and perhaps the author didn't want to support negative inputs anyway. However, the answer contains no disclaimer that clarifies this.

BUG: In the question "Rounding integer division (instead of truncating)" for C ([StackOverflowRound]), OP expresses that they want to divide with rounding to the nearest integer, rather than rounding towards zero.

While the top 11 answers are highly decorated with upvotes, they all overflow for large inputs, use `float` (which has accuracy issues), have the wrong rounding mode, or are plain incorrect.

The first correct solution can only be found at 5 upvotes, by user A Fog:

```
unsigned int rounded_division(unsigned int n, unsigned int d) {
    unsigned int q = n / d;  // quotient
    unsigned int r = n % d;  // remainder
    if (r > d>>1  // fraction > 0.5
    || (r == d>>1 && (q&1) && !(d&1))) { // fraction == 0.5 and odd
        q++;
    }
    return q;
}
```

Unfortunately, this solution does not support negative inputs, and is somewhat branch-heavy. When compiled with Clang, a possible path of execution would go through four conditional jumps depending on `(r > d>>1)`, `(r == d>>1)`, `(q&1)`, and `!(d&1)`, where all but `(r == d>>1)` are extremely unpredictable (basically coin flips). See godbolt.org/z/ax6hr1r53.

BUG: These division functions taken from [BuggyDivisions] have several problems:

```
static int div_floor(int a, int b) {
    return (a ^ b) < 0 && a ? (1 - abs(a)) / abs(b) - 1 : a / b;
}
```

```
static int div_round(int a, int b) {
    return (a ^ b) < 0 ? (a - b / 2) / b : (a + b / 2) / b;
}

static int div_ceil(int a, int b) {
    return (a ^ b) < 0 || !a ? a / b : (abs(a) - 1) / abs(b) + 1;
}
```

- `abs(a)` and `abs(b)` have undefined behavior given `INT_MIN` input
- `div_round` overflows on large inputs in `a - b` and `a + b`
- all functions branch depending on whether the quotient is negative, which may be highly unpredictable

NOTE: See § Appendix A — Reference implementation for proper implementations.

Users sometimes also implement these functions using `std::floor` or `std::ceil`, but this can equally yield incorrect results, and use of floating-point numbers is unnecessary for this task.

§ 2.2. Computing remainders is hard, actually

There exist various use cases (modular arithmetic, integer-to-string conversion, etc.) where we need both the quotient and remainder at the same time. The naive approach to computing the remainder using an integer division function *`divide`* looks something like:

```
int x = /* ... */, y = /* ... */;
int quotient = divide(x, y);
int remainder = x - quotient * y;
```

This approach is only safe when *`divide`*`(x,y)` is `x/y`, i.e. when rounding towards zero.

BUG: The following function, which yields the quotient rounded towards $+\infty$, as well as the remainder, is not safe for large inputs:

```
std::div_t div_rem_ceil(int x, int y) {
    bool quotient_positive = (x < 0) == (y < 0);
    bool has_remainder = x % y != 0;
    int quotient = x / y + int(quotient_positive && has_remainder);
    int remainder = x - y * quotient;
    return { quotient, remainder };
}
```

If we now call `div_rem_ceil(INT_MAX, 2)`, where `INT_MAX` is `2'147'483'647`, then `quotient` is `1'073'741'824`. The multiplication `y * quotient` has undefined behavior because it results in `2'147'483'648`, which cannot be represented as `int`.

Crucially, it does not mean that `div_rem_ceil` cannot be defined for these inputs. The correct quotient is `1'073'741'824`, and the correct remainder is `-1`. It just means that

> for any rounding mode except towards zero, we cannot use the naive formula to obtain a remainder.

In conclusion, the proposal should also include a way to obtain a remainder in tandem with the quotient. Any other design would simply invite bugs where users foolishly assume that the `x - quotient * y` method works for all rounding modes.

3. Design

The design aims of this proposal are to provide *concise*, *simple*, *efficient*, *robust* functions which are useful *in practice*.

> **IMPORTANT:** As a rule of thumb, the proposed functionality should be a drop-in replacement for the various bad implementations that users have written themselves (§2.1. Is this not trivial for the user to do?), and it should take the underlying implementation (§ Appendix A — Reference implementation) into consideration. This rule of thumb influences *every* design choice.

> **NOTE:** You may find a complete list of functions in §5. Try it yourself, §6. Wording, and § Appendix A — Reference implementation.

3.1. Relation to P0105R1

The design somewhat leans on [P0105R1] (which first proposed division functions with custom rounding), but heavily deviates from it. In general, [P0105R1] was a proposal with overly broad scope, and some questionable design choices, such as:

- Making some of the rounding modes conditionally supported, even when providing them isn't particularly difficult.
- Adding rounding modes which optimize for latency or code size, without much motivation or details. The proposal provided no concrete example of how this implementation freedom could be utilized, and I suspect that any implementation would default to rounding to zero, matching the `/` operator anyway.

3.2. Naming

All functions which compute a quotient begin with `div`, and functions which also compute a remainder begin with `div_rem`. Furthermore, the functions include the name of the rounding mode in a format that requires as little prior knowledge as feasible.

3.2.1. Isn't `std::div_*` inconsistent with the existing `std::div`?

It is worth pointing out that there already exist `std::div` functions in the standard library, returning `std::div_t`, which contains the quotient and remainder. The proposal does not aim

for consistency because the facilities are an unnecessary and rarely-used C relic for the most part.

On the contrary, when investigating existing practice in §2.1. Is this not trivial for the user to do?, I found that almost every definition of these differently-rounded division functions uses `div_*` names. This naming scheme is *well-established* and *intuitive* for users.

3.2.2. Should it be `std::div_floor` and `std::div_ceil`?

While the use of names like `floor` and `ceil` is common in various domains, including in `<cmath>`, I do not believe we should perpetuate this design because:

- The scheme does not nicely extend to division with rounding away from zero; there is no established term for that.
- A hypothetical `std::div_round` (for rounding to the nearest integer) would be somewhat perplexing because *all* proposed functions round, just towards different targets.
- The scheme is needlessly hostile towards novices who are not yet familiar with the fact that $\mathrm{floor}(x)$ rounds towards $-\infty$. By comparison, `std::div_to_neg_inf` is self-documenting.

Regardless whether the functions end up called `std::div_floor` or `std::div_to_neg_inf`, the names should remain somewhat brief so they take up a reasonable amount of space in C++ expressions.

3.2.3. Should it be `std::div_to_pos_inf`?

While it looks nicer on paper if `std::div_to_pos_inf` and `std::div_to_neg_inf` have symmetrical names, this does not have any clear technical benefit to the user. `std::div_to_inf` is sufficiently clear, and adding `pos` would merely make the function name longer.

3.2.4. Why is it `std::mod`?

> NOTE: See §3.5. `std::mod`.

Despite the fact that `mod` is somewhat of a misnomer ("remainder" and "modulo" are synonyms), and despite that it is inconsistent with `std::div_rem`, it is arguably the best name because:

- It follows the conventions of Haskell, Ada, CSS, and many more languages, where `mod` rounds towards $-\infty$ (unlike `%`, `rem`, etc.). See §1.1.1. Language support for integer division.
- It matches the notation of the modulo operator in mathematical literature. That is, $-2 \bmod 5 = 3$.

3.3. Interface

The functions generally match the style of similar features in `<numeric>`, like [numeric.sat] or [numeric.ops.gcd]. The design also follows [P3161R4].

EXAMPLE: For rounding to zero, the following functions exist:

```
template<class T>
struct div_result {
    T quotient;
    T remainder;
    friend auto operator<=>(const div_result&, const div_result&) = defau
};

template<class T>
  constexpr div_result<T> div_rem_to_zero(T x, T y) /* not noexcept */;
template<class T>
  constexpr T div_to_zero(T x, T y) /* not noexcept */;
```

It is constrained to accept only signed or unsigned integer types.

3.3.1. Error handling and `noexcept`

These functions are not `noexcept` because they could not have a wide contract; they naturally have undefined behavior for cases like `div_to_zero(x, 0)` or `div_to_zero(INT_MIN, -1)`. Looking at § Appendix A — Reference implementation, it would require additional effort to perform error handling such as throwing exceptions, so it seems best to leave these functions undefined if and only if division is undefined too.

However, an invocation of these functions is not a constant expression for such inputs; i.e. we don't get undefined behavior during constant evaluation. This can even be implemented without a single additional line of code: since we always perform a division in these functions, they naturally get disqualified from being core constant expressions when division is undefiend.

3.3.2. Why no `std::rounding` parameter?

As compared to [P0105R1], I do not propose that the rounding mode is passed as a runtime parameter.

In virtually every case, the rounding mode for an integer division is a fixed choice. This is evidenced by the ten trillion existing uses of the `/` operator which *always* truncate. Also, the implementation of the proposed functions does not lend itself to a runtime parameter:

```
int div_to_zero(int x, int y) {
    return x / y;
}

int div_to_neg_inf(int x, int y) {
    bool quotient_negative = (x ^ y) < 0;
    return (x / y) - int(x % y != 0 && quotient_negative);
}
```

```
int div_away_zero(int x, int y) {
    constexpr auto sgn = [](int z) { return z < 0 ? -1 : 1; };
    int quotient_sign = sgn(x) * sgn(y);
    return (x / y) + int(x % y != 0) * quotient_sign;
}
```

As can be seen, these implementations are substantially different.

> NOTE: While `div_to_neg_inf` could *theoretically* use the `quotient_sign` instead, it requires more operations to compute this positive/negative sign instead of merely checking whether the quotient is negative. This would be an unnecessary performance penalty.
>
> In practice, compilers optimize `sgn(x) * sgn(y) < 0` strictly worse than `(x ^ y) < 0`.

If we now provided a runtime `std::rounding`, the obvious implementation would look like:

```
enum struct rounding { to_zero, to_neg_inf, away_zero, /* ... */ };

int divide(int x, int y, rounding_mode mode) {
    switch (mode) {
    case rounding::to_zero: return __div_to_zero(x);
    case rounding::to_neg_inf: return __div_to_neg_inf(x);
    case rounding::away_zero:  return __div_away_zero(x);
    // ...
    }
    std::unreachable();
}
```

The user can trivially make such an `enum class` and `switch` themselves, if they actually need to. If they don't (which is likely), all we accomplish is making the user write `std::divide(std::rounding::to_neg_inf, x, y)` instead of `std::div_to_neg_inf(x, y)`.

3.3.3. `std::div_result<T>`

As seen in in §3.3. Interface, we define an additional class template `std::div_result`:

```
template<class T>
struct div_result {
    T quotient;
    T remainder;
    friend auto operator<=>(const div_result&, const div_result&) = default;
};
```

This generally matches standard library style, including the active proposal [P3161R4], and including the "legacy types" `std::div_t`, `std::ldiv_t`. That is, we should not use reference output parameters, but return a value.

However, we should also not reuse those legacy types because it is not possible to deduce their "member type" from the class, which would make them clunky in generic code. There also exist no such classes for extended integer types and bit-precise integer types.

`operator<=>` exists because the user may want to compare the quotient and remainder to an existing pair of values in one go, or store results of `std::div_rem` functions in a `std::set`, etc. It would seem arbitrarily limiting to make `std::div_result` incomparable.

3.3.4. Do we really need the `std::div_rem_*` functions?

Yes, because it is non-trivial to compute the remainder without overflow for rounding modes other than towards zero. See §2.2. Computing remainders is hard, actually.

Furthermore, it is relatively cheap to obtain the remainder as a side product of any division function (see § Appendix A — Reference implementation). On extremely feature-starved hardware with no instructions for division and multiplication, computing the remainder via multiplication may be more expensive than producing it "directly" during division (implemented in software).

3.4. Supported rounding modes

It is apparent that the proposal contains quite a lot of rounding modes – more than just the traditional `ceil`, `floor`, `round`. It is therefore valid to ask:

> Do we really need all these rounding modes?

All proposed "top-level rounding modes" have practical applications. For consistency, the same set of "tie-breaking rounding modes" is provided. In any case, the implementation of these modes is all fairly similar, and there is neither any significant implementation cost (§ Appendix A — Reference implementation) nor wording cost (§6. Wording) to having a few extra functions. This would be a much different discussion for floating-point numbers.

I suspect that cherry-picking the "useful" rounding modes out of the proposed ones would devolve into endless discussion, and the design would have an inconsistent feel to it if say, division away from zero was provided, but no division to the nearest integer with tie breaks away from zero.

3.4.1. Rounding to zero

The "trivial" functions `std::div_to_zero` and `std::rem_dividend_sign` exist solely for consistency and enhanced expressiveness. Note that not every has a truncating integer division like C++. For example, the `//` operator in Python rounds towards $-\infty$. A team of developers with primarily Python experience/habits may thus benefit from always expressing rounding mode explicitly with `std::div_to_zero` to avoid confusion.

3.4.2. Rounding to even/odd

Rounding to even/odd integers is mainly useful because it is unbiased. That is, when dividing a large amount of integers, results would not gravitate towards one end due to rounding mode

bias towards zero, $+\infty$, etc. This may be important when operating on fixed-point numbers.

[P0105R1] provides additional motivation and explanation of these rounding modes.

3.4.3. To-nearest-rounding division and tie breaking

Saying "round to nearest" is not clear enough in itself because ties (where the fractional part of the quotient is exactly `.5`) could also be resolved in multiple ways. Any of the "top-level rounding modes" could plausibly be chosen as a "tie-breaking rounding mode" as well, which is what this proposal offers.

3.5. `std::mod`

In addition to the quotient (`std::div_*`) and quotient/remainder functions (`std::div_rem_*`), we also define a single function `std::mod` which specifically computes the remainder of the division rounding towards $-\infty$.

Unlike the other rounding modes, the remainder of division rounding to $-\infty$ is uniquely useful for modular arithmetic because the sign of the remainder is the sign of the divisor. In other words, if we divide by a positive number, the remainder is always positive. This is why it has a dedicated function.

Division rounding to zero, where the sign of the remainder is that of the divisor, already has `%`, so no dedicated function is required. The other rounding modes have more chaotic remainder signs, and thus it is rarely useful to compute the remainder in isolation, not in conjunction with the quotient.

> NOTE: See §3.2.4. Why is it `std::mod`?.

4. Implementation experience

A reference implementation can be found on [GitHub]. A simplified version of that, for educational purposes, can be found at § Appendix A — Reference implementation.

Generally speaking, a portable strategy is to implement all divisions by performing a division with rounding towards zero (`/` and `%`), and then adjusting the quotient and remainder to emulate a different rounding mode.

> EXAMPLE: Just to illustrate the principle, consider how one would round towards $+\infty$ with unsigned operands:

```
unsigned div_to_zero(unsigned x, unsigned y) { // for reference
    return x / y;
}
unsigned div_to_inf(unsigned x, unsigned y) {
    return x / y + (x % y != 0); // + 1 if the remainder is nonzero
}
```

On x86_64, Clang compiles this as follows:

```
div_to_zero:                ; edi = x, esi = y
        mov     eax, edi    ; eax = edi
        xor     edx, edx    ; edx = 0 (clear upper 32 bits of divided)
        div     esi         ; eax = eax / esi, edx = eax % esi
        ret                 ; return eax

div_to_inf:                 ; edi = x, esi = y
        mov     eax, edi    ; eax = edi
        xor     edx, edx    ; edx = 0 (clear upper 32 bits of divided)
        div     esi         ; eax = eax / esi, edx = eax % esi
        cmp     edx, 1      ; b = edx < 1
        sbb     eax, -1     ; eax -= -1 + b
        ret                 ; return eax
```

Note that even on architectures where the remainder is not computed simultaneously with the quotient like for `div`, only one division is necessary; the remainder can be obtained from the quotient.

Consequently, the implementation effort for all of these functions is close to zero. None of them require intrinsics or much architecture-specific knowledge; if any, rounding towards zero is supported in hardware (§1.1.2. Hardware support for integer division), and even if it isn't, `/` has to exist and the other rounding modes can be implemented in terms of it, exactly the same.

§ 5. Try it yourself

If you have JavaScript enabled, you can play around with the following code block.

```
int x = -12;
int y = 5;

// input expression              →  output
double(x) / double(y)            →  -2.4

std::div_to_zero(x, y)           →  -2 // x / y
std::div_away_zero(x, y)         →  -3
std::div_to_inf(x, y)            →  -2
std::div_to_neg_inf(x, y)        →  -3
std::div_to_odd(x, y)            →  -3
std::div_to_even(x, y)           →  -2

std::div_ties_to_zero(x, y)      →  -2
std::div_ties_away_zero(x, y)    →  -2
std::div_ties_to_inf(x, y)       →  -2
std::div_ties_to_neg_inf(x, y)   →  -2
std::div_ties_to_odd(x, y)       →  -2
std::div_ties_to_even(x, y)      →  -2
```

```
x % y                          →  -2 // sign matches x
std::mod(x, y)                 →  3 // sign matches y
```

> NOTE: This demonstration conveniently ignores that `int` has finite size. Under the hood, calculations are performed using JavaScript's `BigInt`.

6. Wording

All changes are relative to [N5008].

6.1. [structure.specifications]

In [structure.specifications] paragraph 3, add a bullet immediately following bullet 3.4:

> - *Preconditions*: conditions that the function assumes to hold whenever it is called; violation of any preconditions results in undefined behavior.
> - *Hardened preconditions*: [...]
> - ***Constant-checked preconditions*: equivalent to a *Preconditions* specification, except that a function call expression that violates the assumed condition is not a core constant expression ([expr.const]).**

Change [structure.specifications] paragraph 4 as follows:

> Whenever the *Effects* element specifies that the semantics of some function `F` are *Equivalent to* some code sequence, then the various elements are interpreted as follows. If `F`'s semantics specifies any *Constraints* or *Mandates* elements, then those requirements are logically imposed prior to the *equivalent-to* semantics. Next, the semantics of the code sequence are determined by the *Constraints*, *Mandates*, *Preconditions*, *Hardened preconditions*, ***Constant-checked preconditions***, *Effects*, *Synchronization*, *Postconditions*, *Returns*, *Throws*, *Complexity*, *Remarks*, and *Error conditions* specified for the function invocations contained in the code sequence. The value returned from `F` is specified by `F`'s *Returns* element, or if `F` has no *Returns* element, a non-`void` return from `F` is specified by the `return` statements ([stmt.return]) in the code sequence. If `F`'s semantics contains a *Throws*, *Postconditions*, or *Complexity* element, then that supersedes any occurrences of that element in the code sequence.

6.2. [version.syn]

Change the synopsis in [version.syn] as follows:

```
#define __cpp_lib_integer_comparison_functions      202002L // also in <util
#define __cpp_lib_integer_division                  20XXXXL // freestanding
```

```
#define __cpp_lib_integer_sequence                  201304L // freestanding,
```

6.3. [bit.pow.two]

Change [bit.pow.two] paragraph 5 as follows:

~~*Preconditions:*~~ ***Constant-checked preconditions:*** *N* is representable as a value of type `T`.

Delete [bit.pow.two] paragraph 8:

~~*Remarks*: A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression ([expr.const]).~~

6.4. [numeric.ops.overview]

Change the synopsis in [numeric.ops.overview] as follows:

```
namespace std {
  […]

  // [numeric.sat], saturation arithmetic
  template<class T>
    constexpr T add_sat(T x, T y) noexcept;
  template<class T>
    constexpr T sub_sat(T x, T y) noexcept;
  template<class T>
    constexpr T mul_sat(T x, T y) noexcept;
  template<class T>
    constexpr T div_sat(T x, T y) noexcept;
  template<class T, class U>
    constexpr T saturate_cast(U x) noexcept;

  // [numeric.int.div], integer division
  template<class T>
  struct div_result {
    T quotient;
    T remainder;
    friend auto operator<=>(const div_result&, const div_result&) = defaul
  };

  template<class T>
    constexpr T div_to_zero(T x, T y);
  template<class T>
    constexpr T div_away_zero(T x, T y);
  template<class T>
    constexpr T div_to_inf(T x, T y);
  template<class T>
    constexpr T div_to_neg_inf(T x, T y);
  template<class T>
    constexpr T div_to_odd(T x, T y);
```

```
  template<class T>
    constexpr T div_to_even(T x, T y);
  template<class T>
    constexpr T div_ties_to_zero(T x, T y);
  template<class T>
    constexpr T div_ties_away_zero(T x, T y);
  template<class T>
    constexpr T div_ties_to_inf(T x, T y);
  template<class T>
    constexpr T div_ties_to_neg_inf(T x, T y);
  template<class T>
    constexpr T div_ties_to_odd(T x, T y);
  template<class T>
    constexpr T div_ties_to_even(T x, T y);

  template<class T>
    constexpr div_result<T> div_rem_to_zero(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_away_zero(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_to_inf(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_to_neg_inf(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_to_odd(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_to_even(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_ties_to_zero(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_ties_away_zero(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_ties_to_inf(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_ties_to_neg_inf(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_ties_to_odd(T x, T y);
  template<class T>
    constexpr div_result<T> div_rem_ties_to_even(T x, T y);

  template<class T>
    constexpr T mod(T x, T y);
}
```

6.5. [numeric.sat.func]

Change [numeric.sat.func] paragraph 9 as follows:

~~*Preconditions:*~~ ***Constant-checked preconditions:*** `y != 0` is `true`.

Delete [numeric.sat.func] paragraph 11:

Remarks: A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression ([expr.const]).

6.6. [numeric.int.div]

Append a new subclause to [numeric.ops], following [numeric.sat]:

Integer division [numeric.int.div]

```
template<class T>
  constexpr T div_to_zero(T x, T y);
```

1 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

2 *Constant-checked preconditions*: `x / y` is well-defined.

3 *Returns*: $\frac{x}{y}$, rounded towards zero.

[*Note*: The result equals `x / y`. — *end note*]

```
template<class T>
  constexpr T div_away_zero(T x, T y);
```

4 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

5 *Constant-checked preconditions*: `x / y` is well-defined.

6 *Returns*: $\frac{x}{y}$, rounded away from zero.

```
template<class T>
  constexpr T div_to_inf(T x, T y);
```

7 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

8 *Constant-checked preconditions*: `x / y` is well-defined.

9 *Returns*: $\frac{x}{y}$, rounded towards positive infinity.

```
template<class T>
  constexpr T div_neg_inf(T x, T y);
```

10 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

11 *Constant-checked preconditions*: `x / y` is well-defined.

12 *Returns*: $\frac{x}{y}$, rounded towards negative infinity.

```
template<class T>
  constexpr T div_to_odd(T x, T y);
```

13 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

14 *Constant-checked preconditions*: `x / y` is well-defined.

15 *Returns*: $\frac{x}{y}$, rounded towards the nearest odd integer.

```
template<class T>
  constexpr T div_to_even(T x, T y);
```

16 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

17 *Constant-checked preconditions*: `x / y` is well-defined.

18 *Returns*: $\frac{x}{y}$, rounded towards the nearest even integer.

```
template<class T>
  constexpr T div_ties_to_zero(T x, T y);
```

19 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

20 *Constant-checked preconditions*: `x / y` is well-defined.

21 *Returns*: $\frac{x}{y}$, rounded towards the nearest integer. If two integers are equidistant, the result is the integer with lower magnitude.

```
template<class T>
  constexpr T div_ties_away_zero(T x, T y);
```

22 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

23 *Constant-checked preconditions*: `x / y` is well-defined.

24 *Returns*: $\frac{x}{y}$, rounded towards the nearest integer. If two integers are equidistant, the result is the integer with greater magnitude.

```
template<class T>
  constexpr T div_ties_to_inf(T x, T y);
```

25 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

26 *Constant-checked preconditions*: `x / y` is well-defined.

27 *Returns*: $\frac{x}{y}$, rounded towards the nearest integer. If two integers are equidistant, the result is the greater integer.

```
template<class T>
  constexpr T div_ties_to_neg_inf(T x, T y);
```

28 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

29 *Constant-checked preconditions*: `x / y` is well-defined.

30 *Returns*: $\frac{x}{y}$, rounded towards the nearest integer. If two integers are equidistant, the result is the lower integer.

```
template<class T>
  constexpr T div_ties_to_odd(T x, T y);
```

31 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

32 *Constant-checked preconditions*: `x / y` is well-defined.

33 *Returns*: $\frac{x}{y}$, rounded towards the nearest integer. If two integers are equidistant, the result is the odd integer.

```
template<class T>
  constexpr T div_ties_to_even(T x, T y);
```

34 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

35 *Constant-checked preconditions*: `x / y` is well-defined.

36 *Returns*: $\frac{x}{y}$, rounded towards the nearest integer. If two integers are equidistant, the result is the even integer.

```
template<class T>
  constexpr div_result<T> div_rem_rounding(T x, T y);
```

37 *Constraints*: `T` is a signed or unsigned integer type ([basic.fundamental]).

38 *Constant-checked preconditions*: `x / y` is well-defined.

39 *Returns*: A result object where

- `quotient` is the quotient q returned by `div_rounding(x, y)` and
- `remainder` is $x - qy$ if `T` is signed, otherwise the integer congruent to $x - qy$ modulo 2^N where N is the width of `T`.

[*Note*: It is possible for `div_rounding(x, y)` to have well-defined behavior even when `x - quotient * y` has undefined behavior.
[*Example*: Assume that `INT_MAX` equals `2'147'483'647`.

```
const auto q_to_zero = INT_MAX / 2;              // q_to_zero is 1'073'
const auto [q, r] = div_rem_to_inf(INT_MAX, 2);  // q is 1'073'741'824 a
int r2 = x - q * 2;                              // This multiplication ha
```

— *end example*] — *end note*]

```
template<class T>
  constexpr T mod(T x, T y);
```

40 *Effects*: Equivalent to `div_rem_to_neg_inf(x, y).remainder`.
[*Note*: The result is negative if and only if `y` is negative and `x` is nonzero. — *end note*]

WARNING: If the mathematical notation in the block above does not render for you, you are using an old browser with no MathML support. Please open the document in a recent version of Firefox or Chrome.

6.7. [simd.bit]

Change [simd.bit] paragraph 4 as follows:

~~*Preconditions*:~~ ***Constant-checked preconditions:*** For every i [...].

Delete [simd.bit] paragraph 6:

Remarks: A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression ([expr.const]).

7. Acknowledgements

I sincerely thank Lawrence Crowl (the author of the predecessor paper [P0105R1]) for reviewing this paper in great detail, providing detailed feedback. The choice to include combined quotient/remainder functions was only made after his feedback.

8. References

[N5008] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-03-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf

[P0105R1] *Lawrence Crowl.* Rounding and Overflow in C++ (2017-02-05)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0105r1.html

[P1889R1] *Alexander Zaitsev.* C++ Numerics Work In Progress (2019-12-27)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1889r1.pdf

[P3161R4] *Tiago Freire.* Unified integer overflow arithmetic (2025-03-26)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3161r4.html

[StackOverflowCeil] Fast ceiling of an integer division in C / C++
https://stackoverflow.com/q/2745074/5740428

[StackOverflowRound] Rounding integer division (instead of truncating)
https://stackoverflow.com/q/2422712/5740428

[BuggyDivisions] *njohnny84.* Rounding Modes For Integer Division
https://blog.demofox.org/2022/07/21/rounding-modes-for-integer-division/

[GitHub] *Jan Schultke.* integer-division GitHub repository
https://github.com/Eisenwave/integer-division

§ Appendix A — Reference implementation

See below a simplified implementation for educational purposes, which works exclusively with `int`. See [GitHub] for the full implementation.

```
#include <type_traits>
#include <compare>

template<class T>
struct div_result {
    T quotient;
    T remainder;
    friend auto operator<=>(const div_result&, const div_result&) = default;
};

template<class T>
constexpr T __sgn2(T x) {
    if constexpr (std::is_signed_v<T>) {
        // Equivalent to: (x >> (width_of_T - 1)) | 1
        return x < 0 ? -1 : 1;
    } else {
        return 1;
    }
}

/// Given a dividend x, divisor y, and quotient offset d (-1, 0, or 1),
/// returns (x / y + d) as the quotient,
/// and a remainder of a division between x and y
/// that would have yielded that quotient.
template<class T>
constexpr div_result<T> __div_rem_offset_quotient(T x, T y, T d) {
    if constexpr (std::is_signed_v<T>) {
        using U = std::make_unsigned_t<T>;
        return {
            .quotient = x / y + d,
            // This is (x % y - d * y),
            // except that we use unsigned int to avoid overflow when y is INT_
            // Due to modular arithmetic rules, when we multiply y with -1,
            // the remainder is congruent to (x % y + y),
            // so simply doing it all with unsigned integers fixes our overflow
            .remainder = T(U(x % y) - U(d) * U(y))
        };
    } else {
        return {
            .quotient = x / y + d,
            .remainder = x % y - d * y
        };
    }
}

// Idea: trivial implementation.
constexpr div_result<int> div_rem_to_zero(int x, int y) {
```

```
    return { .quotient = x / y, .remainder = x % y };
}

constexpr int div_to_zero(int x, int y) {
    return x / y;
}

// Idea: since '/' truncates,
//       we need to increase the quotient magnitude
//       in all cases except when the remainder is zero.
constexpr div_result<int> div_rem_away_zero(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    bool increment = x % y != 0;
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_away_zero(int x, int y) {
    return div_rem_away_zero(x, y).quotient;
}

// Idea: since '/' truncates,
//       the result is one greater than what we want
//       for negative quotients, unless the remainder is zero.
constexpr div_result<int> div_rem_to_inf(int x, int y) {
    bool quotient_positive = (x ^ y) >= 0;
    bool adjust = x % y != 0 && quotient_positive;
    return {
        .quotient  = x / y + int(adjust),
        .remainder = x % y - int(adjust) * y,
    };
}

constexpr int div_to_inf(int x, int y) {
    return div_rem_to_inf(x, y).quotient;
}

// Idea: since '/' truncates,
//       the result is one lower than what we want
//       for positive quotients, unless the remainder is zero.
constexpr div_result<int> div_rem_to_neg_inf(int x, int y) {
    bool quotient_negative = (x ^ y) < 0;
    bool adjust = x % y != 0 && quotient_negative;
    return {
        .quotient  = x / y - int(adjust),
        .remainder = x % y + int(adjust) * y,
    };
}

constexpr int div_to_neg_inf(int x, int y) {
    return div_rem_to_neg_inf(x, y).quotient;
}

// Idea: same as div_away_zero,
//       but we only magnify when this gets us away
```

```cpp
//       from an even integer.
constexpr div_result<int> div_rem_to_odd(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    bool increment = x % y != 0 && x / y % 2 == 0;
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_to_odd(int x, int y) {
    return div_rem_to_odd(x, y).quotient;
}

// Idea: same as div_away_zero,
//       but we only magnify when this gets us away
//       from an odd integer.
constexpr div_result<int> div_rem_to_even(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    bool increment = x % y != 0 && x / y % 2 != 0;
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_to_even(int x, int y) {
    return div_rem_to_even(x, y).quotient;
}

// Idea: same as div_away_zero,
//       but we only magnify when the remainder
//       is greater than abs(y / 2).
constexpr div_result<int> div_rem_ties_to_zero(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    int abs_rem = x % y * __sgn2(x);
    int abs_half_y = y / 2 * __sgn2(y);
    bool increment = abs_rem > abs_half_y;
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_ties_to_zero(int x, int y) {
    return div_rem_ties_to_zero(x, y).quotient;
}

// Idea: same as div_away_zero, but we only magnify when the remainder
//       is greater or equal to abs(y / 2).
//       This is actually somewhat tricky because abs(y / 2) drops one bit of p
//       i.e. the bit indicating .5 or .0 in the number,
//       and (abs(2 * x % y) >= abs(y)) may overflow, so we cannot use that ins
//       However, we can get back that one bit of precision using (y % 2 != 0),
//       which optimizes to (y & 1).
//       When y is even, that bit is zero and we didn't drop any precision anyw
//       When y is odd, there are no exact ties, and we increase the right hand
//       of the comparison to bias more towards truncation instead of magnifica
constexpr div_result<int> div_rem_ties_away_zero(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    int abs_rem = x % y * __sgn2(x);
    int abs_half_y = y / 2 * __sgn2(y);
    bool increment = abs_rem >= abs_half_y + int(y % 2 != 0);
```

```
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_ties_away_zero(int x, int y) {
    return div_rem_ties_away_zero(x, y).quotient;
}

// Idea: same as div_ties_away_zero,
//       but we only magnify on ties when the quotient is positive.
constexpr div_result<int> div_rem_ties_to_inf(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    int abs_rem = x % y * __sgn2(x);
    int abs_half_y = y / 2 * __sgn2(y);
    bool increment = abs_rem >= abs_half_y + int(y % 2 != 0 || quotient_sign <
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_ties_to_inf(int x, int y) {
    return div_rem_ties_to_inf(x, y).quotient;
}

// Idea: same as div_ties_away_zero,
//       but we only magnify on ties when the quotient is negative.
constexpr div_result<int> div_rem_ties_to_neg_inf(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    int abs_rem = x % y * __sgn2(x);
    int abs_half_y = y / 2 * __sgn2(y);
    bool increment = abs_rem >= abs_half_y + int(y % 2 != 0 || quotient_sign >
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_ties_to_neg_inf(int x, int y) {
    return div_rem_ties_to_neg_inf(x, y).quotient;
}

// Idea: same as div_ties_away_zero,
//       but we only magnify on ties when the quotient is even.
constexpr div_result<int> div_rem_ties_to_odd(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
    int abs_rem = x % y * __sgn2(x);
    int abs_half_y = y / 2 * __sgn2(y);
    int quotient = x / y;
    bool increment = abs_rem >= abs_half_y + int(y % 2 != 0 || quotient % 2 !=
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_ties_to_odd(int x, int y) {
    return div_rem_ties_to_odd(x, y).quotient;
}

// Idea: same as div_ties_away_zero,
//       but we only magnify on ties when the quotient is odd.
constexpr div_result<int> div_rem_ties_to_even(int x, int y) {
    int quotient_sign = __sgn2(x) * __sgn2(y);
```

```
    int abs_rem = x % y * __sgn2(x);
    int abs_half_y = y / 2 * __sgn2(y);
    int quotient = x / y;
    bool increment = abs_rem >= abs_half_y + int(y % 2 != 0 || quotient % 2 ==
    return __div_rem_offset_quotient(x, y, int(increment) * quotient_sign);
}

constexpr int div_ties_to_even(int x, int y) {
    return div_rem_ties_to_even(x, y).quotient;
}

// Idea: if the there is a mismatch between the x and y being negative,
//       the result sign would be wrong, and we need to flip it;
//       If they match (i.e. if the quotient is positive),
//       we already have the right result.
//       If the mismatch is caused by the dividend being negative,
//       the remainder is also negative and we should add the (positive) diviso
//       If the mismatch is caused by the divisor being negative,
//       we should add the (negative) divisor to get a negative remainder.
//       In either case, adding the divisor is the right thing to do.
constexpr int mod(int x, int y) {
    bool quotient_negative = (x ^ y) < 0;
    int rem = x % y;
    return rem + y * int(rem != 0 && quotient_negative);
}
```

NOTE: Since integer division occurs unconditionally in these functions, they also trivially satisfy the §6. Wording requirement that invocations of these functions are not constant expressions when the result is not representable.